GroundOne: A Specialist Decision Model for Groundedness Verification in Local Retrieval-Augmented Generation

Francesco Pio De Dominicis · Supervisor: Prof. Paolo Garza
Master's Degree in Computer Engineering, Politecnico di Torino, A.Y. 2025/2026

Thesis summary

Motivation

Retrieval-augmented generation (RAG) gives language models access to private documents, and the whole pipeline can now run on a consumer device, which helps privacy, offline use and cost. Retrieval does not remove hallucinations: answers still contain sentences that the retrieved context does not support or contradicts. Detecting them must happen on every answer, on the same resource-limited machine that hosts the generator. Locality has a precise technical consequence: on a single device reading text is cheap and writing it is expensive. The model reads its input in one forward pass, bounded by compute, but writes one token at a time, each bounded by the memory bandwidth needed to read all the weights; on a laptop a written token costs an order of magnitude more than a read one, and the same asymmetry holds across current local hardware. Verifiers today are either small encoders, which return one fixed score per document–sentence pair, or LLM judges, which write their verdicts token by token and often break the output format. A third family, *decision models*, answers closed questions in one forward pass by reading the probability of each answer option from the logits. Whether a compact decision model specialised in a single task can match LLM judges and encoders, where its advantage comes from, and whether its probabilities are usable, had not been studied in a controlled way on consumer hardware.

GroundOne

GroundOne is a decision model specialised in one decision: for every sentence of a RAG answer, does the retrieved document support it, contradict it, or not mention it? A marker follows every sentence, and the verdicts of all sentences are read from the logits of three answer letters at the markers, in a single forward pass, without generating any text (Figure 1). Two small Qwen2.5 models are fine-tuned with LoRA on the human-annotated answers of RAGTruth, with a class-weighted loss restricted to the three letters. A temperature and a per-class bias fitted on the dev set calibrate the probabilities, and a threshold chosen on the same data turns them into labels that agree with the probabilities shown to the user. Because the instructions and the document come before the answer, their KV cache can be kept and reused exactly when the same sources are verified again. Everything, from dataset construction to training, evaluation and serving, runs on one Apple laptop with MLX and no external service.

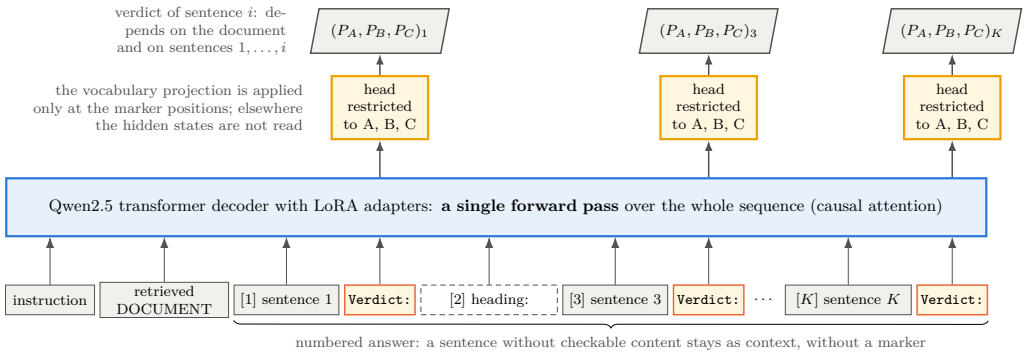


Figure 1: GroundOne's marked prompt: the verdicts of all sentences are read at the `Verdict:` markers in one forward pass.

Data and experimental setup

No label was annotated anew: RAGTruth's span annotations were converted into a three-way label per sentence with rules fixed in advance. Three test sets were built, checked and frozen before any training, with leakage controlled at document level: the RAGTruth test split (in domain), LLM-AggreFact (a broad grounding benchmark) and VitaminC (an out-of-domain fact-checking test). GroundOne is compared with cues that never read the document, the untrained models, LLM judges of the same size with and without probabilities, two generalist decision models (Rizzo-Flow and Liquid AI's d1-3B) and specialised encoders (MiniCheck-RoBERTa, HHEM), all on the same machine, with confidence intervals from a cluster bootstrap.

Results

Specialisation works in the domain it targets. On RAGTruth GroundOne is far more accurate than the same models before training, ranks supported sentences better than the specialised encoders, and in three-way accuracy clearly beats both generalist decision models, including a larger one. On LLM-AggreFact the larger model is on par with MiniCheck, not above it. On VitaminC the gain disappears and a generalist decision model is much stronger: a specialist is not a general fact-checking tool. The readout matters by itself: with the same trained model, choosing among the three admissible letters is clearly more accurate than letting the model generate its verdict, at the same latency, and the gap is largest for the smaller model.

GroundOne is faster than LLM judges of the same size at every answer length, and its advantage grows with the number of sentences: on long answers it is roughly twice as fast as a terse judge and several times faster than a verbose one, always with a valid output. Splitting the time shows why the overall gain is moderate: reading the document and the answer is almost all of GroundOne's time and is paid by every approach, while the deciding part, which the readout replaces, is over an order of magnitude cheaper than writing verdicts. When the document has already been read, as when a rewritten answer is verified again, reusing the KV cache makes verification several times faster for GroundOne but helps the judges much less, because their decoding cannot be reused. GroundOne also uses less energy, although the saving is smaller than the time saving because a long prefill draws more power than decoding. After calibration its probabilities are close to the observed frequencies, while LLM judges asked for the same output give probabilities that barely separate right from wrong decisions. The evaluation runs with no network access and reproducible results, 8-bit weights cost no accuracy, and generator and verifier fit together in one process with plenty of memory to spare.

Question	Finding
In domain (RAGTruth)	Large gain over the untrained models; better ranking than the encoders; clearly above both generalist decision models.
Other benchmarks	On par with MiniCheck on LLM-AggreFact; no gain on VitaminC, where a generalist is stronger.
Readout vs generated verdict	Restricted readout clearly more accurate with the same weights and cost.
Speed vs LLM judges	Faster at every length, more so on long answers; much faster still with the sources already cached.
Probabilities	Well calibrated after a simple post-hoc fit; judges' probabilities nearly uninformative.

Table 1: Main findings, qualitatively; the exact figures are reported in the thesis.

GroundOne Studio: a local application and a case study

A native macOS application, written in Swift with MLX Swift, puts the verifier to work in a complete RAG assistant in which retrieval (SQLite full-text search and small embeddings), generation, verification and a local LLM judge share one process, without network access and well within the memory of the laptop. A conformance test shows identical tokens and practically identical probabilities to the Python implementation. The generator's native answer is shown first; the user

can then verify it with GroundOne alone or together with the LLM judge, inspect the probabilities and the supporting passage of each sentence, and have flagged or selected sentences rewritten from the sources, one short request per sentence and by error type, with a check that rejects a rewrite that merely repeats the flagged sentence. In a case study with injected errors GroundOne flags almost every error, more often than the LLM judge, and targeted rewriting removes most of them: changed numbers and dates are mostly restored from the source, fabricated sentences deleted. Repeating the study with the smaller verifier shows why the three-way verdict matters: it labels changed values as not mentioned rather than contradicted, and the generator then deletes them instead of correcting them. A larger, selectable generator corrects even more errors but is markedly slower and adds more content that the sources do not state. These answers are accepted by GroundOne, not independently verified.

Limitations and future work

The main limits are the narrow training data, through which contradictions outside data-to-text and hallucinations hidden in otherwise copied sentences are learnt only in part, a training recipe not fully robust to the seed, a single platform and model family, and English-only data. Future work includes a more robust recipe (bias correction, gradient clipping, several seeds), training data with paraphrased support and minimal edits, fusing the adapter into the base weights, a llama.cpp port to measure other hardware, per-language adapters, attribution of each verdict to a passage, verification while the answer is being generated, and an independent check of rewritten sentences in the application.